ARQUITETURA DE SOFTWARE E DE DADOS SEGUROS

NOSSA HISTÓRIA

A nossa história inicia com a realização do sonho de um grupo de empresários, em atender à crescente demanda de alunos para cursos de Graduação e Pós-Graduação. Com isso foi criado a nossa instituição, como entidade oferecendo serviços educacionais em nível superior.

A instituição tem por objetivo formar diplomados nas diferentes áreas de conhecimento, aptos para a inserção em setores profissionais e para a participação no desenvolvimento da sociedade brasileira, e colaborar na sua formação contínua. Além de promover a divulgação de conhecimentos culturais, científicos e técnicos que constituem patrimônio da humanidade e comunicar o saber através do ensino, de publicação ou outras normas de comunicação.

A nossa missão é oferecer qualidade em conhecimento e cultura de forma confiável e eficiente para que o aluno tenha oportunidade de construir uma base profissional e ética. Dessa forma, conquistando o espaço de uma das instituições modelo no país na oferta de cursos, primando sempre pela inovação tecnológica, excelência no atendimento e valor do serviço oferecido.

Sumário

INTRODUÇÃO 4

PORQUE SE PENSAR EM SEGURANÇA..... 5

APRENDENDO SOBRE A IMPORTÂNCIA DE UM SOFTWARE SEGURO..... 6

O QUE É IMPORTANTE NO DESIGN DE UM SOFTWARE SEGURO? 7

SEGURANÇA É UM PROCESSO..... 12

QUAL É O PROCESSO DE DESENVOLVIMENTO DE UM SOFTWARE SEGURO?..... 14

BOAS PRÁTICAS DE SEGURANÇA DE SOFTWARE..... 15

A DIFERENÇA DE UM SOFTWARE SEGURO E INSEGURO..... 17

ESTILO ARQUITETURAL 18

A IMPORTÂNCIA DO REÚSO 20

CONSIDERAÇÕES FINAIS..... 23

REFERÊNCIAS 25

INTRODUÇÃO

Como o aprimoramento da modelagem de ameaças afeta o ciclo de vida do desenvolvimento de software

A busca pela transformação digital tem guiado as estratégias de negócios nos dias atuais e, neste contexto, vários pilares da área de TI se destacam, entre eles: o desenvolvimento de software (olhe ao seu redor e pergunte a si mesmo: a que distância você está de um aplicativo de software?), e a segurança da informação, cujo valor reside na redução de riscos (um dos princípios da governança de TI).

Considerando a velocidade com que as práticas estão mudando, o desenvolvimento e o uso de software podem frequentemente parecer mais uma arte do que uma ciência. Conforme as ferramentas e metodologias de software se tornam cada vez mais disponíveis, há uma maior oportunidade de desenvolver softwares, mas também de fraudar os sistemas existentes. Além disso, o software agora é amplamente compartilhado e novas abordagens para reduzir o custo de desenvolvimento, minimizar os riscos da segurança da informação e aumentar a velocidade de entrega estão em constante crescimento e expansão. À medida que as organizações confiam nas tecnologias em evolução, padrões de falha operacional, uso indevido e abuso emergem com mais frequência desde uma variedade de fontes, bem como de práticas internas fracas durante a aquisição ou desenvolvimento de software.

PORQUE SE PENSAR EM SEGURANÇA

Competência tecnológica é fundamento para o sucesso nos negócios. Cadeias produtivas estão cada vez mais integradas. No “chão de fábrica”, as máquinas físicas já têm “gêmeas digitais”. No varejo, as vendas são apoiadas por tecnologia e ocorrem em multicanalidade, com curadoria individual.

Fora das empresas, a tecnologia vem impactando, de maneira evidente, cada vez mais, o dia-a-dia das pessoas. Bens de consumo têm cada vez mais sensores, processadores e conexões com a internet. Produtos tecnológicos são canais por onde consumimos serviços, muitos deles essenciais.

A onipresença de sistemas digitais, tanto na indústria quanto nas rotinas individuais, além de aumentar o impacto percebido de falhas acidentais, os tornam alvos naturais para ataques criminosos. Por isso, segurança é atributo de qualidade cada vez mais importante em arquiteturas modernas. Sistemas inseguros ameaçam empresas, tanto sob aspectos diretamente financeiros e de imagem, e, não raro, a vida das pessoas!

Infelizmente, sistemas dificilmente serão seguros acidentalmente, ou seja, sem projeto apropriado. O acaso, quando se trata de segurança em sistemas, é desfavorável e não dá proteção aos distraídos.

APRENDENDO SOBRE A IMPORTÂNCIA DE UM SOFTWARE SEGURO

Se você quer construir um sistema seguro, você precisa entender de design seguro. Espero que você não comece lendo Secure Software Design, de Richardson e Thies. Enquanto o livro descreve muitas das principais questões em segurança de aplicações de segurança e TI em geral, e algumas ameaças e vulnerabilidades comuns, ele (ironicamente, dado o título) não explica como fazer design de software seguro.

E muito da “informação prática” no livro é no mínimo perigosa, mas não totalmente correta: a seção sobre XSS, por exemplo, que menciona o escape da saída, não explica como fazê-la corretamente, ou que é muito mais importante que ficar limpando a entrada, removendo caracteres desnecessários ou alterando caracteres necessários, mas possivelmente perigosos” (no entanto, você iria fazer isso de forma segura). Ou o principalmente errado: a seção de design de banco de dados seguro- não, “Uma das maneiras mais simples de proteger uma aplicação web de um ataque de injeção de SQL é validar todos os parâmetros de entrada” não é correto, e “Você também deve evitar a dinâmica SQL e usar procedimentos parametrizados armazenados” não está perto o suficiente de ser correto para ser compreendido ou seguido corretamente.

O livro faz aumentar a conscientização sobre as questões de segurança de aplicativos e, logo no início, os autores apontam os leitores para CERT , SANS e OWASP, então não há esperança de que os alunos vão encontrar e utilizar esses recursos em vez de confiar nesse livro.

O QUE É IMPORTANTE NO DESIGN DE UM SOFTWARE SEGURO?

Há muitos erros arquitetônicos e de design básicos que podem comprometer a segurança de um sistema:

Falta algo importante em recursos de segurança, como requisitos de auditoria, privacidade e conformidade de controle de acesso;

Erros técnicos na compreensão e implementação de defesa se segurança contra as “artes do mal”, como criptografia, gerenciamento de segredos e gerenciamento de sessão (você não sabe o suficiente para fazer algo ou para fazê-lo direito);

Incompreensão de responsabilidades arquitetônicas e zonas de confiança, como contar com a validação do lado do cliente ou “Eu pensei que os dados já tinham sido higienizados”;

Deixar a superfície de ataque maior do que tem que ser porque a maioria dos desenvolvedores não entende o que é superfície de ataque de um sistema ou sabe que precisa estar atenta quando mudá-la;

Permitir acesso por padrão, então quando acontece um erro ou alguém se esquece de adicionar a verificação certa no lugar certo, as portas e as janelas são deixadas abertas, e os bandidos podem entrar;

Escolher uma plataforma de desenvolvimento inseguro ou uma pilha de tecnologia, ou um framework, ou uma API, e herdar o design e os erros de codificação de alguém;

Cometer erros estúpidos em fluxos de trabalho de negócios que permitem que os atacantes ignorem as verificações e os limites, e roubem dinheiro ou informações.

Todos esses fatores compreendem o projeto no nível arquitetural e estão diretamente relacionados com a organização do sistema e, portanto, afetam os atributos de qualidade (também chamados de requisitos não funcionais) como desempenho, portabilidade, confiabilidade, disponibilidade, entre outros. Se fizermos uma comparação entre arquitetura de software (caracterizada, por exemplo, pelo estilo em camadas) e arquitetura 'clássica' (relativa à construção de edificações), podemos observar que o projeto arquitetural é determinante para o sucesso do sistema.

A **Tabela 1** destaca aspectos da representação de projeto que captura elementos característicos da arquitetura enquanto que as restrições estão associadas a atributos de qualidade e, portanto, servem como determinantes nas decisões do projeto arquitetural. Por exemplo, embora o uso de múltiplas camadas facilite a manutenção de um sistema de software, também contribui para degradar o desempenho do sistema. Uma tática tem sido reduzir o nível de acoplamento entre componentes para não comprometer o desempenho do sistema. Dessa forma, se adotarmos uma redução no nível de acoplamento dos componentes, eles terão menor necessidade de comunicação entre si, o que resulta num melhor desempenho.

Tabela 1. Comparação de aspectos arquiteturais.

Categorias arquiteturais	**Representações de projeto**	**Restrições**

Arquitetura Clássica	Modelos, desenhos, planos, elevações e perspectivas	Padrão de circulação, acústica, iluminação e ventilação
Arquitetura de Software	Modelos para diferentes papéis, múltiplas visões	Desempenho, confiabilidade, escalonamento e manutenibilidade

Hoje em dia, os processos de engenharia de software requerem o projeto arquitetural de software. Por quê?

- É importante poder reconhecer as estruturas comuns existentes de modo que arquitetos de software (ou engenheiros de software realizando o papel de arquiteto de softwares) possam entender as relações existentes nos sistemas em uso e utilizar esse conhecimento no desenvolvimento de novos sistemas.
- O entendimento das arquiteturas permite aos engenheiros tomarem decisões sobre alternativas de projeto.
- Uma especificação arquitetural é essencial para analisar e descrever propriedades de um sistema complexo, permitindo o engenheiro ter uma visão geral completa do sistema.
- O conhecimento de notações para descrever arquiteturas permite engenheiros comunicarem novos projetos e decisões arquiteturais tomadas a outros membros da equipe.

Cabe destacar que, para que haja o entendimento da arquitetura, faz-se necessário ao engenheiro de software conhecer os estilos arquiteturais existentes, conforme apresentado adiante. As propriedades de cada arquitetura, portanto, são dependentes do estilo arquitetural adotado. Por exemplo, o uso de uma notação padrão como a UML ajuda na representação de componentes e compartilhamento de informações do projeto.

Esses aspectos servem como indicadores de uma maturidade inicial de engenharia de software. Outros aspectos compreendem uso e reuso de soluções existentes no desenvolvimento de novos sistemas. Para tanto, a prototipagem tem sido usada em projetos de natureza inovadora (bem antes da implementação ou aceitação de um produto acontecer).

Além disso, o aumento da complexidade e quantidade de requisitos dos sistemas dificulta cada vez mais atender às restrições de orçamento e cronograma. Atualmente, empresas têm procurado incorporar estratégia de reuso de software, enfatizando o reuso centrado na arquitetura para obter melhores resultados no desenvolvimento de sistemas. Note que a arquitetura de software serve como uma estrutura através da qual se tem o entendimento dos componentes de um sistema e de seus inter-relacionamentos. Em outras palavras, ela define a estrutura do sistema, de modo consistente para implementações, já que está diretamente relacionada aos atributos de qualidade como confiabilidade e desempenho.

A organização dos componentes num sistema de software impacta sobre a qualidade apresentada por ele. Por exemplo, a adoção de uma arquitetura em camadas serve para modularizar o sistema bem como facilitar modificações. Entretanto, um número elevado de camadas (4 ou 5) podem degradar o desempenho do sistema se houver um elevado grau de acoplamento entre os componentes.

Diversos benefícios decorrem da incorporação da arquitetura de software como ‘elemento norteador’ do processo de desenvolvimento de software. Cabe destacar que a arquitetura pode:

- Prover suporte ao reuso – seus componentes definidos e testados podem ser reaproveitados em novas aplicações.

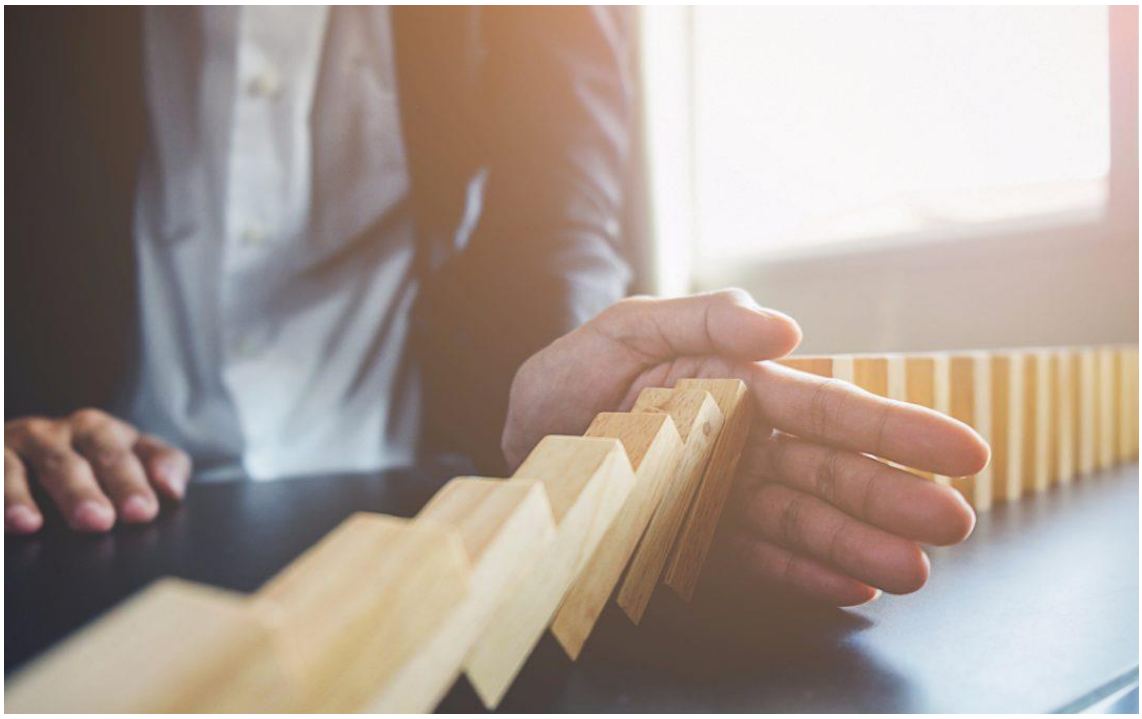
- Servir de base à estimação de custos e gerência do projeto – a existência de uma arquitetura bem definida permite ao gerente de projeto adequadamente alocar tarefas de, por exemplo, implementação de componentes e melhor estimar o tempo e tamanho de equipe necessária para realização de um projeto.

- Servir de base para análise da consistência e dependência – o arquiteto de software pode verificar se a arquitetura de software adotada suporta os atributos de qualidade desejados de modo consistente e avaliar o nível de dependência dos atributos de qualidade em relação à arquitetura. Para tanto, ele faz a análise arquitetural que verifica o suporte oferecido pela arquitetura a um conjunto de atributos de qualidade (como desempenho, portabilidade e confiabilidade).

- Ser utilizada para determinar atributos de qualidade do sistema – o arquiteto de software faz a análise arquitetural a fim de determinar os atributos de qualidade. Trata-se de um processo iterativo.

- Atuar como uma estrutura para atender os requisitos do sistema – a arquitetura ajuda a definir os requisitos funcionais, que compreendem o conjunto de funcionalidades do sistema de software, e requisitos não funcionais (ou atributos de qualidade) que determinam as características visíveis ao usuário como desempenho e confiabilidade.

SEGURANÇA É UM PROCESSO



(Fonte: http://menezeseassociados.com.br/gerenciamento-de-seguranca-de-processo/ **)**

Problemas de segurança têm origem na maldade, inocência, estupidez ou infortúnio humano. As medidas para tornar sistemas mais seguros incluem mecanismos para coibição de ações maliciosas, prevenção de uso inadequado ou descuidado e, finalmente, planos de contingência.

Segurança não “acontece”, tampouco é “ligada” em sistemas. Na prática, ela é garantida por projeto cuidadoso, implantação precisa, manutenção disciplinada e operação monitorada.

Segurança é um processo. (Bruce Schneier)

Sistemas desenvolvidos e mantidos com processos que não dão ênfase para a segurança têm alta probabilidade de serem “vitimados” direta ou indiretamente.

Segurança bem implementada desencoraja e coíbe condutas maliciosas, previne enganos e protege contra infortúnios (afinal, acidentes acontecem). Por isso, diminui desperdícios de tempo e dinheiro, protegendo a privacidade das pessoas e a reputação das marcas.

Os custos relacionados com segurança têm relação com a prevenção de perdas e não com a geração de ganhos. Por isso, embora necessários são sempre questionados. Segurança é um atributo “invisível”, ou seja, só percebido quando há problemas.

Importante destacar que o Brasil, hoje, é segundo colocado mundial na relação de países com maior volume de ataques cibernéticos identificados. Logo, os riscos associados a operações tecnológicas em nosso país são inerentemente mais altos.

Cabe aos arquitetos a responsabilidade de evidenciar os benefícios de priorizar a segurança. Para isso, é útil estimar os custos possíveis na ocorrência “eventos ruins”.

QUAL É O PROCESSO DE DESENVOLVIMENTO DE UM SOFTWARE SEGURO?

Durante o desenvolvimento de um software, os gerentes de TI buscam garantir a entrega de um produto com uma baixa quantidade de bugs no menor tempo possível. No entanto, durante esse processo, não são raros os casos em que gestores ignoram práticas de segurança que evitam, no futuro, vazamento de dados sigilosos. Diante desse cenário, é fundamental que certas rotinas sejam tomadas, ainda que os modelos de criação de software mais tradicionais não sejam focados no desenvolvimento de software seguro.

Peguemos como exemplo o Modelo em Cascata (ou Waterfall Mode, em inglês). Por ser um dos primeiros modelos a serem criados, foi altamente adotado, ainda que possuísse falhas. Suas etapas são:

- verificação dos requerimentos de software e sistema;
- analise e criação da estrutura do software
- desenvolvimento do código do programa;
- teste, depuração e busca por erros;
- instalação, suporte e manutenção de todos os sistemas interligados ao software.

Ainda que algumas versões modificadas do modelo tenham sido criadas, a original peca pela falta de uma etapa em que são implementadas políticas e rotinas de segurança de software. Já modelos como o espiral são focados na capacidade da equipe para desenvolver um programa com um bom aproveitamento do tempo e do esforço de todos os profissionais envolvidos. Dessa forma, é possível eliminar rapidamente elementos que podem atrasar a criação do sistema ou diminuir a produtividade do time de desenvolvedores.

BOAS PRÁTICAS DE SEGURANÇA DE SOFTWARE

A adoção de um modelo de desenvolvimento tradicional não deve ser visto como um motivo para a não implementação de rotinas de segurança. Ao buscar práticas que permitam o desenvolvimento de um software estável e com poucas falhas, o gerente de TI faz com que o seu sistema seja lançado sem falhas graves de segurança. Isso diminui a possibilidade dos dados de segurança do usuário e da sua empresa ficarem vulneráveis.

Em um mundo onde pessoas mal-intencionadas trabalham buscando por brechas que permitam a obtenção de dados pessoais e corporativos, o cuidado com a segurança deve passar por todas as etapas de criação de um software e ser incorporado no cotidiano dos profissionais de TI. Veja abaixo alguns exemplos de como tornar o seu sistema mais seguro:

- Na etapa de classificação de requisitos e formação da equipe de desenvolvimento, é necessário identificar a necessidade de utilização de métodos de criptografia, segurança de usuários, treinamento e, em alguns casos, questões legais envolvendo licenciamento e manipulação de dados;

- Quando a arquitetura do software for projetada, é necessário levar em conta o nível de segurança da arquitetura e da linguagem escolhida, as possíveis vulnerabilidades que o sistema pode apresentar e quando e como os métodos de autenticação e envio de dados seguro serão utilizados;

- Procure oferecer uma documentação com todos os requerimentos de segurança listados para a sua equipe. Além disso, garanta que o seu time trabalhe de acordo com as definições de segurança do projeto, usando as ferramentas da arquitetura e buscando vulnerabilidades da maneira correta;

- Antes de lançar o seu sistema para o público geral, faça testes nos seus métodos de segurança e tente pensar como alguém que deseja comprometer a segurança do sistema. Nessa etapa, é importante garantir que todos os blocos de código que envolvem informações sensíveis não tenham brechas que possam comprometer a segurança do usuário;

- Por fim, quando o produto for finalmente adotado pelo mercado, continue fazendo monitoramento em busca de falhas de segurança. Permita (e incentive) que o seu usuário envie logs com dados de uso e falhas de software. E caso alguma brecha seja encontrada, trabalhe com o seu time para que um patch de segurança seja lançado o mais rápido possível.

A DIFERENÇA DE UM SOFTWARE SEGURO E INSEGURO

A diferença entre um software seguro e inseguro é sutil, onde na maioria das vezes escutamos dos fornecedores que seus sistemas são "seguros". Quase todos os sistemas controlados por software enfrentam ameaças, engenheiros de software devem estar cientes dessas ameaças e projetar sistemas com defesas confiáveis enquanto entregam valor aos clientes. Recentemente a Open Web Application Security Project (OWASP) publicou uma lista indispensável dos Principais riscos de segurança de aplicação Top 10 2017/2018.

(Fonte: https://blog.convisoappsec.com/orientacao-sobre-a-compra-de-um-software-seguro/)

ESTILO ARQUITETURAL

O estilo arquitetural serve para caracterizar a arquitetura de software de um sistema, possibilitando a:

- Identificação de componentes – o arquiteto identifica quais os principais elementos que tem funcionalidades bem definidas como, um componente de cadastro de (informações de) usuários e um componente de autenticação de usuário numa aplicação Web.

- Identificação de mecanismos de interação – a comunicação entre objetos por meio da troca de mensagens constitui uma forma através da qual os componentes de software interagem entre si.

- Identificação de propriedades – o arquiteto pode analisar as propriedades oferecidas por cada estilo baseado na organização dos componentes e nos mecanismos de interação, conforme discutido abaixo.

O estilo arquitetural considera o sistema por completo, permitindo o engenheiro ou arquiteto de software determinar como o sistema está organizado, caracterizando os componentes e suas interações. Em outras palavras, ele determina uma estrutura para todos os componentes do sistema. O estilo arquitetural compreende o vocabulário de componentes e conectores, além da topologia empregada. Mas, você pode estar se questionando: Por que saber o estilo arquitetural é importante?

Os sistemas de grande porte exigem níveis de abstração mais elevados (justamente onde se têm os estilos) que servem de apoio à compreensão do projeto e comunicação entre os participantes do projeto. Ele é determinante no entendimento da organização de um sistema de software.

Mas, o que se ganha em saber o estilo arquitetural? Ele oferece:

- Suporte a atributos de qualidade (ou requisitos não funcionais);
- Diferenciação entre arquiteturas;
- Menos esforço para entender um projeto;
- Reuso de arquitetura e conhecimento em novos projetos.

Conhecer o estilo arquitetural permite ao engenheiro antecipar, por meio da análise (arquitetural), o impacto que o estilo (isto é a classe de organização do sistema) terá sobre atributos de qualidade. Adicionalmente, facilita a comunicação do projeto, além do reuso da arquitetura (da solução).

A caracterização e existência de estilos arquiteturais constituem sinais de amadurecimento da engenharia de software, uma vez que permite ao engenheiro organizar e expressar o conhecimento de um projeto de modo sistemático e útil.

Note que uma forma de codificar conhecimento é dispor de um vocabulário de um conjunto de conceitos (terminologia, propriedades, restrições), estruturas (componentes e conectores) e padrões de uso existentes. Conectores são empregados na interação entre componentes como, tubo (pipe) no estilo tubos e filtros, e mensagens no estilo de objetos.

A IMPORTÂNCIA DO REÚSO

É importante perceber de tudo o que foi apresentado e discutido que um modelo arquitetural oferece suporte ao reuso de várias formas. Por exemplo, pode se ter arquiteturas reusáveis que forneçam a organização e o modelo de coordenação, permitindo seu uso em diversos sistemas. Além disso, podem-se reutilizar componentes, já testados, em mais de um sistema. Disso tudo, o mais importante é o reuso do conhecimento que tem impacto direto na definição da arquitetura de software candidata à solução de um problema (ou sistema).

A ampla variedade de plataformas e utilitários, juntamente com a pressão do mercado para reduzir o tempo de entrega de produtos de software e elevar a produtividade, faz com que o reuso seja apontado como uma das chaves de sucesso para empresas.

O reuso de artefatos (ou componentes) é possível quando o projeto arquitetural está incorporado e orienta o processo de desenvolvimento de software. Isto, como discutido, permite antever os atributos de qualidade que o sistema suporta e também administrar o cronograma de execução do projeto. Portanto, impactando positivamente o reuso e economia do sistema.

O quadro apresentado na **Tabela 2** sumariza as principais características da arquitetura de software e pontos importantes na capacitação e reciclagem de engenheiros e arquitetos de software.

Tabela 2. Características e uso prático da arquitetura de software.

Características de arquitetura de software	Uso prático da arquitetura de software
Constitui um artefato reutilizável	Como um arquiteto de software pode organizar o projeto e código de um sistema
Dispõe de mecanismos de interconexão	Como um arquiteto avalia e implanta arquiteturas de software em sistemas
Oferece um vocabulário de projeto e separa funcionalidades	Como um arquiteto de software atua no processo de desenvolvimento de software
Vincula o projeto a atributos de qualidade	Como um arquiteto avalia a qualidade do código baseada em métricas de produto

Suporta o desenvolvimento baseado em componentes e linha de produto (quando os requisitos são considerados para uma família de sistemas)	Como usar arquitetura como parâmetro para reduzir custos de manutenção e amortizar custos de desenvolvimento

CONSIDERAÇÕES FINAIS

Com a dinamização dos modelos tecnológicos e de gestão, empresas de desenvolvimento e consultoria de software passaram a ter um papel protagonista na busca por falhas de sistema. Se antes isso era conquistado por hackers que buscavam vulnerabilidades para obtenção de senhas e outros dados privados, hoje as desenvolvedoras trabalham na busca por brechas diversas. Para que essa nova postura ganhasse força, requisitos regulatórios foram criados buscando responsabilizar os criadores e não os usuários pelos problemas causados por um software não seguro. Não tenha medo de adaptar as rotinas e protocolos existentes a sua necessidade.

Na era da internet das coisas, a tecnologia tornará a nossa vida melhor dentro e fora do ambiente de trabalho. Nesse cenário em que softwares serão responsáveis pela manipulação de um número cada vez maior de informações, garantir que esses dados estejam seguros é fundamental para as empresas e para os seus usuários. Ao adotar boas práticas que levam ao desenvolvimento de software seguro, o gerente de TI foca os seus recursos na criação e manutenção de um sistema benéfico para a sua empresa e para todos os que dela dependem. Ainda que os desafios estejam cada vez maiores, um sistema que deixe a organização menos vulnerável a ataques criminosos gera menos gastos com contenção e resolução de problemas.

Embora arquitetura de software seja um tema relevante no contexto atual para desenvolvimento de sistemas de software que têm seu foco no reuso e, portanto, consideram tanto o aspecto econômico quanto a produtividade, sua incorporação nos processos de desenvolvimento de software tem sido observada apenas em empresas de grande porte e em poucas de médio porte.

Entretanto, esse cenário começa a mudar dada a crescente necessidade de integração de sistemas a qual tem a arquitetura de software como premissa. Assim, o fator econômico tem sido e será o determinante da sobrevivência da

empresas de software. Novas estratégias de desenvolvimento de sistemas devem ser para o reuso e com reuso (de componentes de software) e o pilar principal dessas estratégias é a arquitetura de software. Há, portanto, uma necessidade premente de formação de capital humano com tais qualificações.

Uma questão que você deve estar se fazendo é: Por que apenas recentemente houve o foco na arquitetura de software?

A resposta é simples: economia e reuso.

Anteriormente não havia forte ênfase na disciplina de engenharia de software, fato este ocorreu com o amadurecimento desta nova área ao longo de toda a década de 1990. Tudo motivou o surgimento de um novo profissional: o arquiteto de software.

REFERÊNCIAS

APOLINÁRIO, Sérgio. **Segurança da informação e a arquitetura de software na era digital.** 2018.

BIRD, Jim, **O que é importante no design de software seguro?** 2013.

FILHO, Antônio. **Arquitetura de Software: Desenvolvimento orientado para arquitetura M S**. 2008.

GRID, Admin. **Quais são os padrões de arquitetura de software?** 2020.

JÚNIOR, Elemar. **Fundamentos para a arquitetura de sistemas seguros**. Capítulo 10 v1. 2021.